

Sprawozdanie zadań algorytmicznych

Spis treści

1. Wyznacznik Macierzy
2. Liczby Pierwsze
3. Liczby Czworacze
4. Bisekcja Wielomian
5. Monte Carlo

Zadanie 1 – Wyznacznik macierzy

Program służy do obliczania wyznacznika macierzy kwadratowej po podaniu jej rozmiaru oraz wszystkich elementów. Użytkownik wpisuje jedną liczbę określającą wielkość macierzy, np. 2 dla macierzy 2x2, 3 dla macierzy 3x3 itd.

Program sprawdza poprawność danych wejściowych. Jeśli użytkownik wpisze literę albo naciśnie Enter bez podania liczby, program wyświetli komunikat błędu i poprosi o ponowne wpisanie wartości. Rozmiar macierzy musi mieścić się w zakresie od 1 do 10.

Po wprowadzeniu wszystkich elementów program wyświetla wpisaną macierz oraz obliczony wyznacznik. Dla macierzy 1x1 i 2x2 wynik jest liczony bezpośrednio, a dla większych macierzy program wykorzystuje rekurencyjne rozwinięcie Laplace'a, tworząc mniejsze macierze pomocnicze.

Screen działania programu / wynik skryptu

```
piotr@Piotr-Zenbook MINGW64 ~/Desktop/go-algorytmy (master)
• $ go run ./zadanie1_wyznacznik_macierzy
=== Wyznacznik macierzy kwadratowej ===
Podaj rozmiar od 1 do 10, a następnie elementy macierzy.

Podaj rozmiar macierzy: 3

--- Uzupełnianie macierzy ---
A[1][1]: 2
A[1][2]: 5
A[1][3]: 2
A[2][1]: 6
A[2][2]: 7
A[2][3]: 8
A[3][1]: 9
A[3][2]: 2
A[3][3]: 1

--- Wprowadzona macierz ---
|      2      5      2 |
|      6      7      8 |
|      9      2      1 |

Wyznacznik macierzy wynosi: 210
```

Kod źródłowy

```
package main
```

```

import (
    "bufio"
    "fmt"
    "os"
    "strconv"
)

const maksRozmiar = 10

func main() {
    scanner := bufio.NewScanner(os.Stdin)

    fmt.Println("=== Wyznacznik macierzy kwadratowej ===")
    fmt.Printf("Podaj rozmiar od 1 do %d, a nastepnie elementy macierzy.\n\n", maksRozmiar)

    size := wczytajRozmiar(scanner)
    matrix := make([][]float64, size)

    fmt.Println("\n--- Uzupełnianie macierzy ---")
    for row := range matrix {
        matrix[row] = make([]float64, size)
        for col := range matrix[row] {
            matrix[row][col] = wczytajLiczbe(scanner, fmt.Sprintf("A[%d][%d]: ", row+1, col+1))
        }
    }

    fmt.Println("\n--- Wprowadzona macierz ---")
    wypiszMacierz(matrix)
    fmt.Printf("\nWyznacznik macierzy wynosi: %.10g\n", wyznacznik(matrix))
}

func wczytajRozmiar(scanner *bufio.Scanner) int {
    for {
        size, err := strconv.Atoi(wczytajTekst(scanner, "Podaj rozmiar macierzy: "))
        if err != nil {
            fmt.Println("Bład: wpisz liczbe calkowita, np. 2 dla macierzy 2x2.")
        } else if size < 1 || size > maksRozmiar {
            fmt.Printf("Bład: rozmiar musi byc w zakresie od 1 do %d.\n", maksRozmiar)
        } else {
            return size
        }
    }
}

func wczytajLiczbe(scanner *bufio.Scanner, prompt string) float64 {
    for {
        number, err := strconv.ParseFloat(wczytajTekst(scanner, prompt), 64)
        if err == nil {
            return number
        }
        fmt.Println("Bład: wpisz liczbe, np. 4 albo 3.5.")
    }
}

func wczytajTekst(scanner *bufio.Scanner, prompt string) string {
    for {
        fmt.Print(prompt)
        if !scanner.Scan() {

```

```

fmt.Println("\nBłąd: zakończono wprowadzanie danych przed uzupełnieniem programu.")
os.Exit(1)
}
if text := scanner.Text(); text != "" {
    return text
}
fmt.Println("Błąd: wartość nie może być pusta.")
}
}

func wyznacznik(matrix [][]float64) float64 {
    size := len(matrix)
    if size == 1 {
        return matrix[0][0]
    }
    if size == 2 {
        return matrix[0][0]*matrix[1][1] - matrix[0][1]*matrix[1][0]
    }

    result, sign := 0.0, 1.0
    for col, value := range matrix[0] {
        result += sign * value * wyznacznik(mniejsza(matrix, col))
        sign *= -1
    }
    return result
}

func mniejsza(matrix [][]float64, skippedCol int) [][]float64 {
    result := make([][]float64, 0, len(matrix)-1)
    for row := 1; row < len(matrix); row++ {
        minorRow := make([]float64, 0, len(matrix)-1)
        for col, value := range matrix[row] {
            if col != skippedCol {
                minorRow = append(minorRow, value)
            }
        }
        result = append(result, minorRow)
    }
    return result
}

func wypiszMacierz(matrix [][]float64) {
    for _, row := range matrix {
        fmt.Println("")
        for _, value := range row {
            fmt.Printf(" %10.10g", value)
        }
        fmt.Println("")
    }
}

```

Zadanie 2 – Liczby pierwsze

Program wczytuje od użytkownika jedną liczbę naturalną n i wypisuje wszystkie liczby pierwsze nie większe od n . Do wyznaczania liczb pierwszych używa sita Eratostenesa.

Jeżeli użytkownik poda błędne dane albo liczbę mniejszą od 1, program wyświetli komunikat:
Podaj poprawne dane.

Screen działania programu / wynik skryptu

```
piotr@Piotr-Zenbook MINGW64 ~/Desktop/go-algorytmy (master)
$ go run ./zadanie2_liczby_pierwsze
Podaj liczbę:
30
2
3
5
7
11
13
17
19
23
29
```

Kod źródłowy

```
package main

import "fmt"

func main() {
    var n int
    if _, err := fmt.Scan(&n); err != nil || n < 1 {
        fmt.Println("Podaj poprawne dane")
        return
    }

    sito := make([]bool, n+1)
    for i := 2; i*i <= n; i++ {
        if !sito[i] {
            for j := i * i; j <= n; j += i {
                sito[j] = true
            }
        }
    }

    for i := 2; i <= n; i++ {
        if !sito[i] {
            fmt.Println(i)
        }
    }
}
```

Zadanie 3 – Liczby czworacze

Program wczytuje od użytkownika jedną liczbę naturalną n i wypisuje wszystkie liczby czworacze mniejsze od n . Liczby czworacze to czwórki liczb pierwszych w postaci $p, p+2, p+6, p+8$.

Do wyznaczania liczb pierwszych program używa sita Eratostenesa, a potem sprawdza, czy dla danej liczby p również $p+2, p+6$ i $p+8$ są pierwsze.

Jeżeli użytkownik poda błędne dane albo liczbę mniejszą od 1, program wyświetli komunikat:

Podaj poprawne dane.

Screen działania programu / wynik skryptu

```
piotr@Piotr-Zenbook MINGW64 ~/Desktop/go-algorytmy (master)
$ go run ./zadanie3_liczby_czworacze
Podaj liczbę: 10000
5 7 11 13
11 13 17 19
101 103 107 109
191 193 197 199
821 823 827 829
1481 1483 1487 1489
1871 1873 1877 1879
2081 2083 2087 2089
3251 3253 3257 3259
3461 3463 3467 3469
5651 5653 5657 5659
9431 9433 9437 9439
```

Kod źródłowy

```
package main

import "fmt"

func main() {
    var n int
    if _, err := fmt.Scan(&n); err != nil || n < 1 {
        fmt.Println("Podaj poprawne dane")
        return
    }

    sito := make([]bool, n+1)
    for i := 2; i*i <= n; i++ {
        if !sito[i] {
            for j := i * i; j <= n; j += i {
                sito[j] = true
            }
        }
    }

    for p := 2; p+8 < n; p++ {
        if !sito[p] && !sito[p+2] && !sito[p+6] && !sito[p+8] {
            fmt.Println(p, p+2, p+6, p+8)
        }
    }
}
```

Zadanie 4 – Bisekcja dla wielomianu 5. stopnia

Program wczytuje od użytkownika współczynniki wielomianu:

$$f(x) = ax^5 + bx^4 + cx^3 + dx^2 + ex + f$$

oraz przedział, na którym ma szukać miejsc zerowych.

Program prosi kolejno o:

- a
- b

- c
- d
- e
- f
- przedział w postaci: początek i koniec

Następnie:

- wyznacza punkty krytyczne pochodnej,
- dzieli przedział na mniejsze fragmenty,
- stosuje metodę bisekcji tam, gdzie występuje zmiana znaku,
- wypisuje wszystkie znalezione miejsca zerowe w podanym przedziale.

Jeżeli dane są błędne albo współczynnik $a = 0$, program wyświetli komunikat:

Podaj poprawne dane.

Jeżeli w podanym przedziale nie ma miejsc zerowych, program wyświetli komunikat:

Brak miejsc zerowych w podanym przedziale.

Screen działania programu / wynik skryptu

```
piotr@Piotr-Zenbook MINGW64 ~/Desktop/go-algorytmy (master)
$ go run ./zadanie4_bisekcja_wielomian5
Podaj a: 1
Podaj b: -1
Podaj c: -7
Podaj d: 1
Podaj e: 6
Podaj f: 0
Podaj przedział (początek koniec): -3 4
Znalezione miejsca zerowe:
x1 = -2.000000
x2 = -1.000000
x3 = 0.000000
x4 = 1.000000
x5 = 3.000001
```

Kod źródłowy

```
package main
```

```
import (
    "fmt"
    "math"
    "sort"
)
```

```
const (
    scanStep      = 0.001
    epsilon       = 0.000001
    duplicateDistance = 0.0001
    maxIterations = 1000
)
```

```
func main() {
    współczynniki, lewy, prawy, err := wczytajDane()
    if err != nil || współczynniki[0] == 0 {
        fmt.Println("Podaj poprawne dane")
        return
    }
}
```

```

if lewy > prawy {
    lewy, prawy = prawy, lewy
}

if lewy == prawy {
    fmt.Println("Podaj poprawne dane")
    return
}

miejscaZerowe := znajdzMiejscaZeroweWielomianu(wspolczynniki, lewy, prawy)
if len(miejscaZerowe) == 0 {
    fmt.Println("Brak miejsc zerowych w podanym przedziale")
    return
}

fmt.Println("Znalezione miejsca zerowe:")
for i, miejsceZerowe := range miejscaZerowe {
    fmt.Printf("x%d = %.6f\n", i+1, miejsceZerowe)
}
}

func wczytajDane() ([]float64, float64, float64, error) {
    wspolczynniki := make([]float64, 6)
    etykiety := []string{"a", "b", "c", "d", "e", "f"}

    for i, etykieta := range etykiety {
        fmt.Printf("Podaj %s: ", etykieta)
        if _, err := fmt.Scan(&wspolczynniki[i]); err != nil {
            return nil, 0, 0, err
        }
    }
}

var lewy, prawy float64
fmt.Print("Podaj przedzial (poczonek koniec): ")
if _, err := fmt.Scan(&lewy, &prawy); err != nil {
    return nil, 0, 0, err
}

return wspolczynniki, lewy, prawy, nil
}

func znajdzMiejscaZeroweWielomianu(wspolczynniki []float64, lewy, prawy float64) []float64 {
    miejscaZerowePochodnej := skanujMiejscaZerowe(obliczPochodna(wspolczynniki), lewy, prawy)
    punkty := append([]float64{lewy, prawy}, miejscaZerowePochodnej...)

    sort.Float64s(punkty)
    punkty = usunPowtorzeniaZPosortowanych(punkty)

    miejscaZerowe := make([]float64, 0)

    for _, punkt := range punkty {
        if math.Abs(obliczWartoscWielomianu(wspolczynniki, punkt)) <= epsilon {
            miejscaZerowe = dolaczJesliUnikalne(miejscaZerowe, punkt)
        }
    }

    for i := 0; i < len(punkty)-1; i++ {
        lewyPunkt := punkty[i]
        prawyPunkt := punkty[i+1]
    }
}

```

```

if prawyPunkt-lewyPunkt <= epsilon {
    continue
}

lewaWartosc := obliczWartoscWielomianu(wspolczynniki, lewyPunkt)
prawaWartosc := obliczWartoscWielomianu(wspolczynniki, prawyPunkt)

if lewaWartosc*prawaWartosc < 0 {
    miejsceZerowe := bisekcja(wspolczynniki, lewyPunkt, prawyPunkt)
    miejscaZerowe = dolaczJesliUnikalne(miejscaZerowe, miejsceZerowe)
}
}

sort.Float64s(miejscaZerowe)
return miejscaZerowe
}

func skanujMiejscaZerowe(wspolczynniki []float64, lewy, prawy float64) []float64 {
    miejscaZerowe := make([]float64, 0)
    dlugoscPrzedzialu := prawy - lewy
    liczbaKrokow := int(math.Ceil(dlugoscPrzedzialu / scanStep))

    for i := 0; i < liczbaKrokow; i++ {
        poczatek := lewy + float64(i)*scanStep
        koniec := lewy + float64(i+1)*scanStep

        if koniec > prawy {
            koniec = prawy
        }

        wartoscPoczatku := obliczWartoscWielomianu(wspolczynniki, poczatek)
        wartoscKonca := obliczWartoscWielomianu(wspolczynniki, koniec)

        if math.Abs(wartoscPoczatku) <= epsilon {
            miejscaZerowe = dolaczJesliUnikalne(miejscaZerowe, poczatek)
        }

        if math.Abs(wartoscKonca) <= epsilon {
            miejscaZerowe = dolaczJesliUnikalne(miejscaZerowe, koniec)
        }

        if wartoscPoczatku*wartoscKonca < 0 {
            miejsceZerowe := bisekcja(wspolczynniki, poczatek, koniec)
            miejscaZerowe = dolaczJesliUnikalne(miejscaZerowe, miejsceZerowe)
        }
    }

    return miejscaZerowe
}

func bisekcja(wspolczynniki []float64, lewy, prawy float64) float64 {
    lewaWartosc := obliczWartoscWielomianu(wspolczynniki, lewy)
    prawaWartosc := obliczWartoscWielomianu(wspolczynniki, prawy)
    poprzedniSrodek := lewy

    for i := 0; i < maxIterations; i++ {
        srodek := (lewy + prawy) / 2
        wartoscSrodka := obliczWartoscWielomianu(wspolczynniki, srodek)

```



```

if math.Abs(wartoscSrodka) <= epsilon || math.Abs(srodek-poprzedniSrodek) <= epsilon {
    return srodek
}

if lewaWartosc*wartoscSrodka < 0 {
    prawy = srodek
    prawaWartosc = wartoscSrodka
} else {
    lewy = srodek
    lewaWartosc = wartoscSrodka
}

if math.Abs(prawy-lewy) <= epsilon || math.Abs(prawaWartosc-lewaWartosc) <= epsilon {
    return (lewy + prawy) / 2
}

poprzedniSrodek = srodek
}

return (lewy + prawy) / 2
}

func obliczWartoscWielomianu(wspolczynniki []float64, x float64) float64 {
    wynik := 0.0
    for _, wspolczynnik := range wspolczynniki {
        wynik = wynik*x + wspolczynnik
    }
    return wynik
}

func obliczPochodna(wspolczynniki []float64) []float64 {
    stopien := len(wspolczynniki) - 1
    pochodna := make([]float64, 0, stopien)

    for i := 0; i < stopien; i++ {
        wykladnik := float64(stopien - i)
        pochodna = append(pochodna, wspolczynniki[i]*wykladnik)
    }

    return pochodna
}

func usunPowtorzeniaZPosortowanych(wartosci []float64) []float64 {
    if len(wartosci) == 0 {
        return wartosci
    }

    unikalne := []float64{wartosci[0]}
    for i := 1; i < len(wartosci); i++ {
        if math.Abs(wartosci[i]-unikalne[len(unikalne)-1]) > duplicateDistance {
            unikalne = append(unikalne, wartosci[i])
        }
    }

    return unikalne
}

func dolaczJesliUnikalne(miejscaZerowe []float64, miejsceZerowe float64) []float64 {

```

```

for _, istniejace := range miejscaZerowe {
    if math.Abs(istniejace-miejsceZerowe) <= duplicateDistance {
        return miejscaZerowe
    }
}

return append(miejscaZerowe, miejsceZerowe)
}

```

Zadanie 5 – Całkowanie metodą Monte Carlo

Program oblicza wartość całki oznaczonej funkcji:

$$f(x) = |\sin(x) + \sin(2x) + \sin(4x) + \sin(8x)|$$

na przedziale $[0, 2\pi]$ metodą Monte Carlo.

Program prosi użytkownika o:

- epsilon – dokładność obliczeń (np. 0.000001)

Działanie programu

Metoda Monte Carlo polega na losowaniu punktów wewnątrz prostokąta otaczającego wykres funkcji i szacowaniu pola powierzchni pod krzywą na podstawie proporcji punktów, które się pod nią znalazły.

Krok 1 – Wyznaczenie prostokąta

Funkcja $f(x)$ spełnia warunek $0 \leq f(x) \leq 4$, dlatego prostokąt ma wymiary:

- $x \in (0, 2\pi)$ – oś pozioma
- $y \in (0, 4)$ – oś pionowa
- pole prostokąta = $2\pi \times 4 = 8\pi$

Krok 2 – Losowanie punktów

Program losuje punkt $P(x, y)$ o współrzędnych:

- x – liczba losowa z przedziału $(0, 2\pi)$
- y – liczba losowa z przedziału $(0, 4)$

Następnie sprawdza warunek: czy punkt leży pod wykresem funkcji, tzn. $y \leq f(x)$.

- Jeśli tak: licznik L1 jest zwiększany o 1
- Jeśli nie: licznik L2 jest zwiększany o 1

Krok 3 – Szacowanie całki

Po każdym wylosowanym punkcie wartość całki jest szacowana wzorem:

$$\int_0^{2\pi} f(x) dx \approx 8\pi \times L1 / (L1 + L2)$$

Krok 4 – Warunek stopu

Program kończy działanie, gdy różnica między kolejnymi szacowaniami jest mniejsza od epsilon:

$$|calka(N+1) - calka(N)| < epsilon$$

Minimalnie generowanych jest 1000 punktów, żeby uniknąć przedwczesnego zakończenia.

Przykładowe uruchomienie

Podaj epsilon: 0.000001

Wynik calki: 7.443840

Liczba punktów: 7443841

Screen działania programu / wynik skryptu

```
piotr@Piotr-Zenbook MINGW64 ~/Desktop/go-algorytmy (master)
$ go run ./zadanie5_monte_carlo
Podaj epsilon: 0.0000001
Wynik całki: 7.440308
Liczba punktów: 74403079
```

Kod źródłowy

```
package main

import (
    "fmt"
    "math"
    "math/rand"
    "time"
)

func f(x float64) float64 {
    return math.Abs(math.Sin(x) + math.Sin(2*x) + math.Sin(4*x) + math.Sin(8*x))
}

func main() {
    var epsilon float64
    fmt.Print("Podaj epsilon: ")
    fmt.Scan(&epsilon)

    rng := rand.New(rand.NewSource(time.Now().UnixNano()))

    // Prostokąt: x ∈ (0, 2π), y ∈ (0, 4), pole = 8π
    rectangleArea := 8 * math.Pi

    var L1, L2 int64 // L1: pod krzywą, L2: nad krzywą
    var prevIntegral float64
    var currentIntegral float64
    const minPoints = 1000

    for {
        x := rng.Float64() * 2 * math.Pi
        y := rng.Float64() * 4

        if y <= f(x) {
            L1++
        } else {
            L2++
        }

        total := L1 + L2
```

```
currentIntegral = rectangleArea * float64(L1) / float64(total)

if total >= minPoints && math.Abs(currentIntegral-prevIntegral) < epsilon {
    break
}

prevIntegral = currentIntegral
}

fmt.Printf("Wynik całki: %.6f\n", currentIntegral)
fmt.Printf("Liczba punktów: %d\n", L1+L2)
}
```